

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	Tamil Elakiya S	Reg. No.:	192524443
Date:	3/08/2026	Duration:	60 minutes

Code of Conduct

I Tamil Elakiya S (192524443) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tamil Elakiya S

Digital farmer Marketplace for Agricultural Product Trading

1. Analyze System Requirements

1. Analyze System Requirements

1.1 Introduction

Before designing the software architecture, it is essential to analyze the system requirements thoroughly. Requirement analysis helps in understanding the overall purpose of the system, identifying the needs of stakeholders, and determining the functional and non-functional requirements. A proper analysis ensures that the proposed architecture is capable of supporting the expected functionalities while maintaining performance, security, scalability, and reliability.

For the Digital Farmer Marketplace, the architecture should support secure online trading, efficient communication between farmers and buyers, seamless payment processing, and easy system maintenance.

1.2 Study the Given Problem Statement and Identify the System Objectives

The Digital Farmer Marketplace aims to provide an online platform where farmers can directly sell their agricultural products to buyers without relying on intermediaries. The existing agricultural trading process often involves middlemen, leading to reduced profits for farmers and increased prices for consumers. Additionally, traditional methods lack transparency, efficient communication, and proper record management.

The proposed system addresses these challenges by creating a centralized digital marketplace that enables farmers to list products, buyers to purchase them online, administrators to monitor platform activities, and delivery partners to ensure timely transportation of goods.

System Objectives

The main objectives of the proposed system are:

- Develop a secure and user-friendly online marketplace for agricultural products.
- Connect farmers directly with buyers to eliminate unnecessary intermediaries.
- Ensure transparent pricing and fair trading practices.
- Maintain digital records of users, products, and transactions.
- Improve accessibility for farmers across different regions.
- Reduce food wastage through faster product sales.

1.3 Analyze the Functional Requirements Influencing the Software Architecture

Functional requirements determine the services that the architecture must support.

The important functional requirements include:

- User Registration and Login
- Role-Based Authentication
- Farmer Profile Management
- Buyer Profile Management
- Product Search and Filtering
- Shopping Cart Management
- Order Placement
- Secure Payment Processing
- Order Tracking
- Delivery Management
- Notifications
- Review and Rating System
- Complaint Handling
- Administrator Dashboard
- Report Generation

1.4 Analyze the Non-Functional Requirements Influencing the Software Architecture

Non-functional requirements define the quality attributes that the software architecture should satisfy.

The proposed system should provide:

- Fast response time during product searching and order processing.
- Secure storage of user credentials and payment information.
- High reliability to minimize system failures.
- Easy usability for farmers with basic digital literacy.
- Scalability to support increasing users and products.
- Maintainability for future updates and enhancements.
- High availability with minimum downtime.
- Compatibility across desktops, tablets, and smartphones.
- Data integrity through proper validation and backup mechanisms.

1.5 Identify Key Quality Attributes

The Digital Farmer Marketplace must satisfy several software quality attributes to ensure long-term success.

- **Scalability**

The architecture should support increasing numbers of users, products, and transactions without reducing system performance.

- **Maintainability**

The system should be modular, allowing developers to modify or upgrade individual components without affecting the entire application.

- **Reliability**

The platform should provide consistent and accurate services with minimal failures, ensuring that user transactions and product information remain reliable.

- **Availability**

The system should remain operational throughout the day, allowing users to access marketplace services whenever required.

- **Performance**

The software should provide fast page loading, efficient product searching, and quick transaction processing even during peak usage.

- **Security**

The system should protect user information, payment details, and transaction records using authentication, authorization, encryption, and secure communication protocols.

2. Select a Suitable Software Architecture

2.1 Introduction

Software architecture defines the overall structure of a software system and describes how different components interact with each other. Selecting the appropriate architecture is an important step because it directly affects the system's performance, scalability, security, maintainability, and future expansion.

For the Digital Farmer Marketplace, a Hybrid Architecture combining Client–Server Architecture and Model–View–Controller (MVC) Architecture is selected. This architecture effectively supports multiple users, secure communication, modular development, and easy maintenance.

2.2 Selected Architecture

Hybrid Architecture (Client–Server + MVC)

The proposed system follows a **Client–Server architecture**, where multiple clients (farmers, buyers, and administrators) access services through a centralized server. Within the server, the application follows the **Model–View–Controller (MVC)** pattern to separate the user interface, business logic, and data management.

Client Layer

The client layer consists of users accessing the application through web browsers or mobile devices. Different users interact with the system according to their roles.

- Farmers manage products and orders.
- Buyers search products and place orders.
- Administrators monitor and manage the system.

Application Server

The application server processes user requests, validates data, executes business logic, manages user authentication, and communicates with the database. It also integrates with external services such as payment gateways and notification systems.

Database Layer

The database securely stores all information related to users, products, orders, payments, delivery details, and transaction history. It ensures data consistency and supports efficient retrieval of information.

External Services

The architecture integrates external services such as:

- Online Payment Gateway
- Email/SMS Notification Service
- Delivery and Logistics Services

2.3 Why Hybrid (MVC + Client–Server) Architecture is Suitable

The Hybrid Architecture is selected because it combines the advantages of both Client–Server and MVC architectures.

Separation of Responsibilities

The MVC pattern separates the application into three independent components:

- **Model** manages data and business rules.
- **View** provides the user interface.
- **Controller** handles user requests and coordinates between the Model and View.

This separation improves code organization and simplifies development.

Supports Multiple Users

The Client–Server architecture allows multiple farmers, buyers, and administrators to access the system simultaneously while maintaining secure communication.

Easy Maintenance

Since each module is developed independently, updates or bug fixes can be made without affecting the entire application.

Better Security

Sensitive operations such as authentication, payment processing, and transaction management are handled on the server, reducing security risks.

Scalable Architecture

The architecture supports future growth by allowing additional servers, databases, or services to be integrated without redesigning the entire system.

Efficient Data Management

All user information, products, orders, and payments are stored in a centralized database, ensuring data consistency and easier management.

2.4 Comparison with Other Architectural Styles

Architecture	Advantages	Limitations	Suitability
Layered Architecture	Simple and modular	Less flexible for large web applications	Moderate
Client–Server Architecture	Supports multiple users and centralized data	Limited separation of internal logic	Good
MVC Architecture	Easy maintenance, modular design	Requires client-server support for deployment	Very Good
Microservices Architecture	Highly scalable and independent services	Complex deployment and management	Suitable for very large enterprise systems
Event-Driven Architecture	Efficient asynchronous processing	More complex implementation	Not necessary for this project
Serverless Architecture	Reduces server management	Vendor dependency and execution limits	Less suitable
Hybrid (MVC + Client–Server)	Combines modularity, scalability, security, and maintainability	Slightly more design effort	Best Choice

2.5 Advantages of the Selected Architecture

The selected architecture provides several benefits:

- Supports multiple users simultaneously.
- Provides secure communication between clients and the server.

- Simplifies software maintenance and future enhancements.
- Enables modular development using MVC principles.
- Supports integration with payment gateways and delivery services.
- Improves system scalability and performance.
- Ensures centralized data management.
- Facilitates efficient testing and debugging.

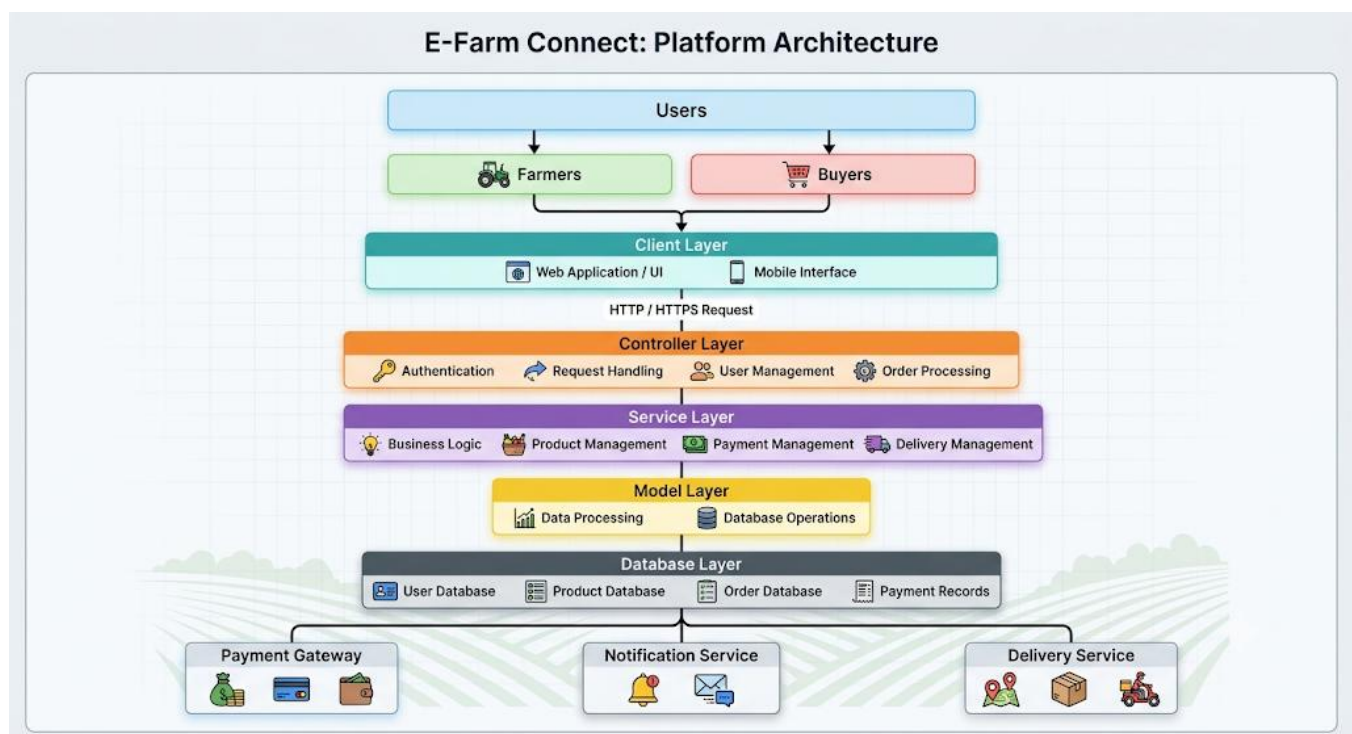
3. Draw the Software Architecture Diagram

3.1 Introduction

The software architecture diagram represents the overall structure of the Digital Farmer Marketplace system. It shows the major components, layers, external services, databases, and communication flow between different parts of the system.

The proposed architecture follows a **Hybrid Client–Server and MVC architecture**, where users interact with the application through the client layer, requests are processed by the application server, and data is stored securely in the database layer.

3.2 Software Architecture Diagram



3.3 Explanation of Architecture Layers

1. Client Layer

The Client Layer represents the interface through which users interact with the Digital Farmer Marketplace.

Responsibilities:

- Provides user-friendly interfaces.
- Accepts user inputs.
- Displays products, orders, and transaction details.
- Allows farmers and buyers to access marketplace services.

Users:

- Farmers
- Buyers
- Administrators

Examples of client functionalities:

- Farmer uploading products.
- Buyer searching products.
- Admin monitoring transactions.

2. Controller Layer

The Controller Layer acts as the communication bridge between the user interface and the business logic.

Responsibilities:

- Receives requests from users.
- Validates user inputs.
- Controls application workflow.
- Sends appropriate responses to users.

Examples:

- Processing login requests.
- Handling product search requests.
- Managing order placement requests.

3. Service Layer

The Service Layer contains the core business logic of the application.

Responsibilities:

- Implements marketplace operations.
- Handles rules and validations.
- Processes transactions.
- Manages communication between different modules.

Major services include:

User Service

- Registration
- Authentication
- Profile management

Product Service

- Product listing
- Inventory updates
- Product searching

Order Service

- Order creation
- Order tracking
- Order status updates

Payment Service

- Payment verification
- Transaction processing

4. Model Layer

The Model Layer manages the application's data structure and communication with the database.

Responsibilities:

- Defines data objects.
- Performs database operations.
- Maintains data consistency.

Examples:

- User model
- Product model
- Order model
- Payment model

5. Database Layer

The Database Layer provides centralized storage for all system information.

Stored Data:

User Database

Contains:

- Farmer details
- Buyer details
- Administrator information

Product Database

Contains:

- Product name
- Category
- Price
- Quantity
- Availability

Order Database

Contains:

- Order details
- Customer information
- Delivery status

Payment Database

Contains:

- Transaction details
- Payment status
- Payment history

6. External System Integration

The marketplace interacts with external systems to improve functionality.

Payment Gateway

Responsible for secure online transactions.

Functions:

- Process payments.
- Verify transaction status.
- Generate payment confirmation.

Notification Service

Provides real-time communication.

Functions:

- Order confirmation alerts.
- Payment notifications.
- Delivery updates.

Delivery Service

Manages transportation of products.

Functions:

- Assign delivery requests.
- Update delivery status.
- Confirm product delivery.

3.4 Communication Flow of the System

The overall workflow of the system is as follows:

1. A farmer or buyer accesses the application through the client interface.
2. The request is sent to the Controller Layer through secure communication.
3. The Controller validates the request and forwards it to the appropriate Service Module.
4. The Service Layer performs required business operations.
5. The Model Layer communicates with the database to store or retrieve information.

3.5 Advantages of the Proposed Architecture

The architecture provides several benefits:

- Clear separation between user interface, business logic, and data management.
- Easy modification of individual modules.
- Supports multiple users simultaneously.
- Improves security through centralized authentication.
- Enables integration with external services.

4. Explain Components and Their Interactions

4.1 Introduction

The Digital Farmer Marketplace consists of multiple software components that work together to provide a complete agricultural trading platform. Each component has a specific responsibility, and communication between these components ensures smooth functioning of the system.

The architecture follows a modular approach where each component performs a separate task. This improves maintainability, reduces complexity, and allows future enhancements without affecting the complete system.

4.2 Description of Architectural Components and Their Responsibilities

1. User Interface Component

Purpose

The User Interface (UI) component provides interaction between users and the system. It allows farmers, buyers, and administrators to access marketplace services through a web or mobile interface.

Responsibilities

- Provide registration and login interfaces.
- Display available agricultural products.
- Allow farmers to upload product details.
- Allow buyers to search and purchase products.
- Display order and payment information.
- Provide dashboards based on user roles.

Interaction

The UI sends user requests to the Controller Layer and displays responses received from the server.

Example:

- Farmer uploads product → UI sends product details to the server.
- Buyer searches product → UI displays matching products from the database.

2. Authentication and User Management Component

Purpose

This component manages user accounts and ensures secure access to the platform.

Responsibilities

- User registration.

- Login authentication.
- Password management.
- Role-based access control.
- Profile management.

Interaction

When a user enters login credentials:

1. The request is sent from the UI.
2. The Controller validates the request.
3. Authentication service verifies user details from the database.
4. Access permission is provided based on user role.

3. Product Management Component

Purpose

The Product Management Component manages agricultural product information provided by farmers.

Responsibilities

- Add new products.
- Update product details.
- Delete unavailable products.
- Manage product quantity.
- Store product images.
- Maintain product availability.

Interaction

The farmer provides product information through the interface. The information is processed by the service layer and stored in the Product Database.

Example:

Farmer → Product Module → Database → Marketplace Display

4. Marketplace and Search Component

Purpose

This component helps buyers discover and purchase agricultural products.

Responsibilities

- Product searching.
- Category filtering.

- Price comparison.
- Display product information.
- Manage shopping cart.

Interaction

When buyers search for products:

1. Search request is sent to the Controller.
2. Product Service processes the request.
3. Database retrieves matching products.
4. Results are displayed to the buyer.

5. Order Management Component

Purpose

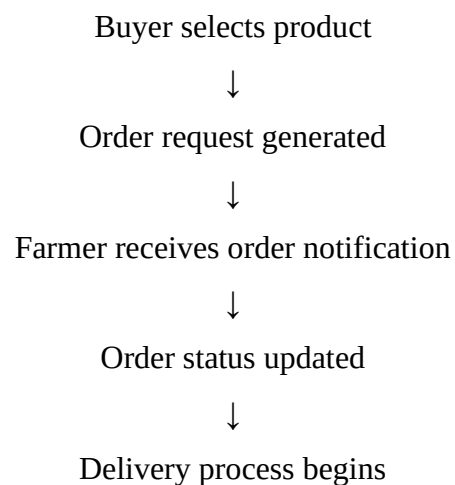
The Order Management Component handles the complete buying and selling process.

Responsibilities

- Create new orders.
- Store order details.
- Update order status.
- Manage order history.
- Connect farmers and buyers.

Interaction

The order process works as follows:



4.4 Data Exchange Between Components

Source Component	Destination Component	Data Exchanged
User Interface	Controller	User requests and inputs
Controller	Service Layer	Processed requests
Service Layer	Model Layer	Data operations
Model Layer	Database	Data storage/retrieval
Payment Module	Payment Gateway	Transaction details
Delivery Module	Delivery Partner	Delivery information
Database	User Interface	Product, order, and account details

5. Discuss Quality Attributes

5.1 Introduction

Quality attributes define the non-functional characteristics that determine the effectiveness, reliability, and long-term success of a software system. A good software architecture should not only provide required functionalities but also ensure that the system is secure, scalable, maintainable, and available for users.

The proposed Hybrid Client–Server + MVC Architecture addresses important quality attributes required for the Digital Farmer Marketplace, including scalability, security, performance, reliability, maintainability, and availability.

5.2 Scalability

Scalability refers to the ability of a system to handle increasing numbers of users, products, and transactions without affecting performance.

How the Architecture Supports Scalability

The proposed architecture supports scalability through:

- **Modular design:** Separate modules for users, products, orders, payments, and administration allow individual components to be expanded independently.
- **Database scalability:** The database can be upgraded to handle increasing product information, user records, and transaction history.
- **Server expansion:** Additional application servers can be added when user traffic increases.
- **Cloud deployment support:** The system can be deployed on cloud platforms to dynamically increase resources based on demand.

5.3 Security

Security ensures that user information, payment details, and system data are protected from unauthorized access and cyber threats.

How the Architecture Supports Security

The proposed architecture improves security through:

- **Authentication and Authorization:** Users are verified before accessing system features.
- **Role-Based Access Control:** Farmers, buyers, and administrators have different access permissions.
- **Encrypted Password Storage:** User passwords are securely stored using encryption techniques.
- **Secure Payment Processing:** Payment information is handled through trusted payment gateways.
- **Data Protection:** Database access is controlled to prevent unauthorized modifications.

5.4 Performance

Performance represents the ability of the system to respond quickly and efficiently to user requests.

How the Architecture Supports Performance

The architecture improves performance through:

- **Layer separation:** Different responsibilities are handled by separate layers, reducing system complexity.
- **Efficient database operations:** Proper database design improves data retrieval speed.
- **Optimized request handling:** The Controller Layer manages user requests efficiently.

6. Justify Architectural Decisions

6.1 Introduction

Selecting an appropriate software architecture is an important decision that influences the overall success of a software system. The architecture should satisfy current system requirements while also supporting future expansion, integration, and maintenance.

For the Digital Farmer Marketplace for Agricultural Product Trading, the Hybrid Client–Server + MVC Architecture has been selected because it provides a balanced approach between simplicity, scalability, security, and maintainability.

6.2 Rationale Behind Selecting the Architecture

The following factors justify the selection of the Hybrid Client–Server and MVC architecture.

1. Separation of Responsibilities

The MVC pattern divides the application into three major components:

- **Model:** Handles data storage and database operations.
- **View:** Manages user interface and presentation.
- **Controller:** Handles user requests and application flow.

2. Supports Multiple User Roles

The Digital Farmer Marketplace involves different users such as:

- Farmers
- Buyers
- Administrators
- Delivery partners

3. Improved Security Management

Since important operations are handled on the server side, sensitive information can be protected effectively.

Security mechanisms include:

- User authentication.
- Role-based access control.
- Data encryption.
- Secure payment integration.

4. Easier Maintenance and Updates

The modular structure allows developers to modify individual components without affecting the entire system.

For example:

- Updating payment services does not require changing product management.
- Improving the user interface does not affect database operations.

5. Support for Future Expansion

The selected architecture allows additional features to be integrated in the future.

Possible enhancements include:

- AI-based product recommendations.
- Crop price prediction.
- Weather-based farming suggestions.
- Mobile application support.
- Data analytics dashboards.
- Blockchain-based transaction tracking.

6.3 Support for Future Enhancements

The selected architecture provides a strong foundation for future improvements.

1. AI Integration

Machine learning modules can be added for:

- Demand prediction.
- Crop price forecasting.
- Personalized recommendations.

2. Mobile Application Development

The same backend services can support Android and iOS applications.

3. Advanced Analytics

Data analysis modules can provide:

- Sales trends.
- Farmer performance reports.
- Market analysis.

4. IoT Integration

Future versions can integrate IoT devices for:

- Smart farming data collection.
- Soil monitoring.
- Crop condition analysis.

6.4 Long-Term Maintainability

The architecture supports long-term maintenance through:

- Modular component design.
- Clear separation of functionality.
- Easy testing and debugging.
- Flexible database management.
- Support for technology upgrades.